

Module M05.C05 — Arbitrer un outil par situation : taille, fréquence, compétences, gouvernance

Analyse de données appliquée & Business Intelligence
M05_C05_arbitrer_un_outil_par_situation.md · document de production

Table des matières

MODULE M05.C05 — ARBITRER UN OUTIL PAR SITUATION : TAILLE, FRÉQUENCE, COMPÉTENCES, GOUVERNANCE	3
1. Objectifs du chapitre	3
2. Pourquoi cette notion est importante	3
3. Explication simple	4
4. Vocabulaire essentiel	4
5. Cours approfondi — la grille de décision	5
5.1 Dire « non » à Excel au-delà de 100 000 lignes	5
5.2 Dire « pas encore » à Python quand l'équipe ne maîtrise pas	6
5.3 Les temps d'exécution mesurés sur les trois tailles réelles	6
5.4 La grille de décision (le tableau de décision complet)	6
5.5 L'extrapolation au million de lignes (étiquetée)	7
6. Exemple concret — le verdict du fil rouge sur les trois moteurs	7
6.1 Mesure du verdict (mars 2025)	7
6.2 La rejouabilité : avril et mai 2025	8
6.3 Le verdict du projet M05.P	8
7. Démonstration pas à pas — comment choisir, en 4 questions	8
7.1 Combien de lignes ?	8
7.2 Quelle fréquence de mise à jour ?	9
7.3 Quelles compétences dans l'équipe ?	9
7.4 Quelle gouvernance ?	9
8. Erreurs fréquentes	9
9. Bonnes pratiques professionnelles	10
10. Exercice guidé — choisir l'outil pour un cas concret (30 min, /10)	10
11. Exercices autonomes	11
12. Correction détaillée	11
13. Mini-projet M05.P5 — « La justification du choix d'outil pour le projet M05.P » (1 h)	12
14. Boîte à outils du chapitre	12
15. Résumé du chapitre	12
16. À retenir	13
17. Évaluation formative (auto-correction, 8 min)	13

Module M05.C05 — Arbitrer un outil par situation : taille, fréquence, compétences, gouvernance

Outils comparés : Power Query (Microsoft 365), SQL DuckDB 1,5,5, pandas 2,2,3. Durée indicative : 4 h. Niveau : N2.

L'idée du chapitre. M05 a installé **trois moteurs** (Power Query, SQL DuckDB, pandas) qui exécutent les mêmes 10 opérations du C01. Lequel choisir pour un fichier donné ? La réponse n'est pas « le plus rapide » ni « le plus moderne » : c'est **une décision qui dépend de quatre facteurs** — la taille du fichier, la fréquence de mise à jour, les compétences de l'équipe, la gouvernance des données. Le chapitre installe la **grille de décision** (tableau du §3), mesure les **temps d'exécution** sur les trois tailles réelles du socle (6 884 / 120 000 / 243 360 lignes), extrapole au **million de lignes**, et démontre le **verdict du fil rouge** : mars, avril et mai 2025 donnent la même table dans les trois moteurs — c'est la **robustesse**, pas la vitesse, qui distingue le bon outil.

Base de travail — les trois tailles réelles du socle, mesurées sur l'atelier de référence. Le verdict est mesuré, pas extrapolé : les temps ci-dessous sont les **médianes** de 3 exécutions sur la machine virtuelle de référence (pandas 2,2,3, DuckDB 1,5,5, machine virtuelle Linux 4 Go). L'extrapolation à un million de lignes est étiquetée « estimation, à vérifier sur un fichier plus gros » (règle 3 du §3 du plan M05).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **comparer** les trois moteurs (Power Query, SQL DuckDB, pandas) sur les **4 critères** du choix : taille, fréquence, compétences, gouvernance ;
- **lire** le tableau de décision du §3 et l'appliquer à un cas concret ;
- **mesurer** les temps d'exécution sur les trois tailles réelles (6 884 / 120 000 / 243 360 lignes) ;
- **extrapoler** au million de lignes en étiquetant le résultat comme « estimation » ;
- **dire « non » à Excel** quand le fichier dépasse 100 000 lignes (la règle de la section §5.1) ;
- **dire « pas encore » à Python** quand l'équipe ne maîtrise pas pandas (la règle de la section §5.2) ;
- **rejouer** le verdict du fil rouge (mars / avril / mai 2025) sur les trois moteurs et vérifier l'empreinte sha256.

2. Pourquoi cette notion est importante

- Vous recevrez un fichier qui n'est pas le vôtre, dans un format qui n'est pas le vôtre, avec une équipe qui n'est pas la vôtre. La question « quel outil » ne se pose pas en termes absolus : elle se pose en termes de **contraintes locales**. Ce chapitre installe la grille qui rend la décision réversible : si le critère « taille » change, on change d'outil ; si le critère « compétences » change, on change d'outil.
- Le **tableau de bord** du magasin (M03) consomme la table propre de M05. Si la table est fautive, le tableau est faux, et la décision qui suit est fautive. **Le choix de l'outil n'est pas une question de goût** : c'est une question de **garantie** sur la qualité de la table.

- **L'extrapolation à un million de lignes** est l'épreuve de vérité : un outil qui marche sur 6 884 lignes peut s'effondrer à un million. Le chapitre donne les **temps mesurés** sur 6 884 / 120 000 / 243 360 lignes, et l'**estimation** au million — étiquetée comme telle. C'est la discipline d'honnêteté du module (règle 3 du §3 du plan M05).
- Les **trois moteurs sont équivalents** sur le verdict (mars = 6 884 lignes, total 464 096 003 FCFA, 101 enrichies, empreinte sha256 identique). La différence est sur la forme, pas sur le fond : le pipeline est portable, et c'est **la garantie de qualité** qui justifie l'investissement du module.
- « **Non** » à **Excel** : sur un fichier > 100 000 lignes, Excel est lent, fragile (il peut planter à l'ouverture), et **non reproductible** (le collègue ne peut pas rejouer le pipeline sans le classeur). C'est l'objet du §5.1.
- « **Pas encore** » à **Python** : pandas est puissant, mais c'est un **langage de programmation**. Une équipe qui ne maîtrise pas les notions de base (variable, fonction, type) ne peut pas maintenir un notebook pandas. C'est l'objet du §5.2.

3. Explication simple

Le tableau de décision en **4 colonnes × 3 lignes** : chaque colonne est un critère (taille, fréquence, compétences, gouvernance), chaque ligne est un moteur (Power Query, SQL DuckDB, pandas). La cellule contient le verdict (« oui », « non », « avec réserve »). La lecture se fait **en ligne** (quand un critère est limitant, c'est lui qui tranche) ou **en colonne** (quel moteur pour un profil donné).

Profil	Moteur	Pourquoi
Analyste solo, fichiers < 100 000 lignes, mise à jour mensuelle	Power Query	Gratuit, déjà installé, parlant la même langue que les collègues non techniques.
Équipe data, fichiers > 100 000 lignes, mise à jour quotidienne	SQL DuckDB	Le plus rapide, le plus portable, le plus reproductible. Nécessite Python ou CLI.
Data scientist, exploration, statistiques avancées	pandas	Le plus riche en méthodes, le plus flexible. Nécessite Python et la maîtrise des DataFrames.

Le **tableau complet** est dans le §5.4.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Taille — Size	Le nombre de lignes du fichier (et accessoirement de colonnes). Le seuil critique est ≈ 100 000 lignes pour Excel, ≈ 10 millions pour DuckDB.	Compter la taille du fichier en octets (Mo) au lieu de lignes : un CSV de 28 Mo peut faire 243 360 lignes (comme notre brut) ou 50 000 lignes (CSV dense). C'est le nombre de lignes qui décide, pas les octets.
Fréquence Frequency	— La périodicité de la mise à jour : mensuelle (bulletin de paie), quotidienne (caisse), temps réel (capteurs). Le pipeline doit rejouer sans intervention humaine.	Coder le pipeline « pour ce fichier » sans paramétrer : si le fichier du mois suivant a un nom différent, le pipeline casse. La règle du module est toujours paramétrer (cf. C02 §5.13).
Compétences Skills	— La maîtrise de l'équipe : tableur (Excel), SQL (DuckDB), Python (pandas). L'outil choisi doit être maintenable par l'équipe.	Imposer un outil que l'équipe ne maîtrise pas : le pipeline est livré, mais personne ne peut le maintenir — c'est l'échec à 6 mois.
Gouvernance Governance	— Les règles de l'entreprise sur les données : qui peut voir quoi, qui peut modifier quoi, où sont les sauvegardes. Le pipeline doit respecter ces règles.	Mettre un CSV sur un partage réseau ouvert : un collègue peut le modifier à la main, et le pipeline ré-exécute produit une table différente de celle attendue.

Français — English	Définition simple	Piège à éviter
Reproductibilité — Reproducibility	Capacité à rejouer le pipeline à l'identique sur le même fichier, sans intervention humaine.	Coder le pipeline « à la main » sans le sauvegarder dans un fichier : si l'analyste part en vacances, personne ne peut rejouer.
Verdict — Verdict	La mesure qui prouve que le pipeline est correct : l'empreinte sha256 de la table propre.	Faire confiance au verdict « ça compile » : un pipeline peut tourner et produire une table fausse. La règle est empreinte sha256 identique entre les trois moteurs.
Estimation — Estimate	Une mesure extrapolée à partir de mesures réelles, étiquetée comme telle. L'extrapolation au million de lignes est une estimation, pas une mesure.	Publier une estimation comme une mesure : le lecteur ne sait pas qu'il lit une extrapolation, et la décision qui suit est risquée.

Définition. La **grille de décision** (tableau du §5.4) est l'outil central du chapitre : 4 critères × 3 outils, avec un verdict par cellule. La lecture se fait **en colonne** (quel moteur pour un profil donné) ou **en ligne** (quand un critère est limitant, c'est lui qui tranche). La règle est la grille est une aide, pas une règle absolue : le vrai critère est le pipeline tourne-t-il dans l'équipe ?

Définition. Le **critère limitant** est le critère qui, seul, tranche le choix de l'outil. Si le fichier fait 750k lignes, le critère « taille » est limitant (Excel exclu) ; le critère « compétences » est secondaire. Si l'équipe ne fait pas de Python, le critère « compétences » est limitant (pandas exclu). La règle est identifier le critère limitant en premier.

Définition. Le **scheduler** est l'outil qui exécute le pipeline à intervalles réguliers (cron sous Linux, Planificateur de tâches sous Windows, Airflow pour les pipelines complexes). C'est ce qui rend la mise à jour quotidienne automatique : le scheduler ré-exécute le pipeline chaque matin à 7h, sans intervention humaine. Sans scheduler, la mise à jour est manuelle et **non reproductible**.

Définition. La **vitesse dépend de la taille** : un outil rapide sur 6 884 lignes peut être lent sur 1 million, et inversement. pandas est rapide sur les petites tailles (lecture et parsing en mémoire) ; DuckDB est rapide sur les grandes tailles (traitement vectorisé et parallèle). L'extrapolation linéaire n'est **pas toujours vraie** : sur certaines opérations (les jointures), DuckDB est 30 × plus rapide, et l'écart grandit avec la taille. La règle est toujours mesurer sur la taille réelle.

5. Cours approfondi — la grille de décision

5.1 Dire « non » à Excel au-delà de 100 000 lignes

Excel est **excellent** pour les fichiers < 100 000 lignes : il est visuel, immédiat, et le collègue non technique peut l'utiliser. Au-delà de 100 000 lignes, **trois problèmes** :

1. **Lenteur** : un TCD sur 500 000 lignes met 30 secondes à s'ouvrir ; sur 1 million, il peut planter. La machine virtuelle de l'atelier (4 Go de RAM) bloque à ≈ 700 000 lignes.
2. **Fragilité** : un fichier `.xlsx` ouvert par deux personnes en même temps est verrouillé ; un classeur avec 50 colonnes × 100 000 lignes met 30 secondes à s'enregistrer.
3. **Non-reproductibilité** : le collègue qui ouvre le classeur dans 6 mois ne voit pas comment la table a été produite (les étapes manuelles ne sont pas documentées).

La **règle** est ne pas dépasser 100 000 lignes dans Excel. Au-delà, on bascule sur SQL DuckDB ou pandas (qui sont aussi gratuits, mais qui nécessitent Python ou CLI).

Attention. La règle « 100 000 lignes » est **indicative** : un fichier avec beaucoup de texte (les montants « 1 200 FCFA » par exemple) bloque plus tôt qu'un fichier avec des entiers. La règle absolue est si Excel met plus de 10 secondes à ouvrir le fichier, il est trop gros.

5.2 Dire « pas encore » à Python quand l'équipe ne maîtrise pas

pandas est **le plus puissant** des trois outils (méthodes riches, exploration facile, statistiques intégrées). Mais c'est un **langage de programmation**, pas un outil cliquable. Une équipe qui ne maîtrise pas les notions de base (variable, fonction, type, indentation) **ne peut pas maintenir** un notebook pandas : le code tourne par chance, et la première erreur (« NameError: name 'df' is not defined ») bloque tout.

La **règle** est ne pas imposer pandas à une équipe qui ne fait pas de Python. Les alternatives :

- **Power Query** pour les équipes bureautiques (Excel Microsoft 365, déjà sur le poste).
- **SQL DuckDB** pour les équipes data (Python est aussi nécessaire pour orchestrer, mais le pipeline lui-même est en SQL, plus facile à maintenir).
- **Une formation Python** (M08) avant d'introduire pandas — c'est l'objet du §4 du plan M05.

Conseil professionnel. Si l'équipe hésite entre pandas et SQL DuckDB, **commencer par SQL** : la syntaxe est plus stable, le pipeline est plus facile à lire, et la transition vers pandas est plus naturelle que l'inverse.

5.3 Les temps d'exécution mesurés sur les trois tailles réelles

Le tableau ci-dessous donne les **temps médians** (en millisecondes) sur 3 exécutions du pipeline complet (import → conversion → coupe à la date → dédoublonnage par rang), mesurés le 19/09/2026 sur l'atelier de référence (pandas 2,2,3, DuckDB 1,5,5, machine virtuelle Linux 4 Go).

Taille (lignes)	pandas (ms)	DuckDB (ms)	Ratio (pandas / DuckDB)
6 884 (mars 2025)	22	283	0,08 (pandas 13 × plus rapide)
120 000 (fenêtre projet)	328	578	0,57 (pandas 1,8 × plus rapide)
243 360 (brut complet)	667	321	2,08 (DuckDB 2 × plus rapide)

L'enseignement : sur les **petites tailles**, pandas est plus rapide (DuckDB pâtit du chargement initial du CSV). Sur les **grandes tailles**, DuckDB prend l'avantage (2 × plus rapide sur 243 360 lignes). L'écart **grandit avec la taille**.

5.4 La grille de décision (le tableau de décision complet)

Critère	Power Query	SQL DuckDB	pandas
Taille < 100 000 lignes	☐ Idéal	☐ Correct dimensionné	(sur- ☐ Correct (sur-dimensionné)
Taille 100 000 — 1 million	☐ Lent / fragile	☐ Idéal	☐ Correct (plus lent que DuckDB)
Taille > 1 million	☐ Impossible	☐ Idéal	☐ Possible (attention RAM)
Mise à jour mensuelle	☐ Paramétrable (MoisCible)	☐ Vue paramétrée	☐ Constante en haut du notebook
Mise à jour quotidienne	☐ Planifiable (Power Automate)	☐ Scheduler (cron)	☐ Scheduler (cron)

Critère		Power Query	SQL DuckDB	pandas
Mise à jour temps réel		❑ Pas adapté	❑ Adapté (DuckDB + Kafka)	❑ Adapté (DuckDB + pandas)
Compétences bureautique	Excel	/	⚠ Nécessite Python ou CLI	❑ Nécessite Python
		❑ Idéal		
Compétences SQL		⚠ Concepts SQL visibles	❑ Idéal	⚠ Concepts SQL embarqués
Compétences Python		❑ Pas de Python	⚠ Python pour orchestrer	❑ Idéal
Gouvernance centralisée		⚠ Power uniquement	❑ Adapté (schéma SQL)	⚠ Nécessite Python + lock
Gouvernance fichier)	locale (1	❑ Adapté	❑ Adapté	❑ Adapté

Lecture : pour un fichier < 100 000 lignes avec une équipe bureautique, **Power Query est idéal**. Pour un fichier > 100 000 lignes avec une équipe data, **SQL DuckDB est idéal**. Pour l’exploration statistique avec un data scientist, **pandas est idéal**. **Excel est exclu** au-delà de 100 000 lignes, **Python est exclu** sans maîtrise préalable.

5.5 L’extrapolation au million de lignes (étiquetée)

L’extrapolation linéaire donne les temps suivants au million de lignes (sur l’atelier de référence) :

Moteur	Temps au million (estimé)
pandas	≈ 2,9 secondes (mesuré sur 1M lignes générées par duplication du brut : 2 936 ms)
DuckDB	≈ 0,9 secondes (mesuré sur 1M lignes générées par duplication du brut : 899 ms)
Power Query	non mesuré (Excel n’a pas pu ouvrir le fichier de 1M lignes)

L’enseignement : au million de lignes, **DuckDB est ≈ 3 × plus rapide que pandas**. Power Query **n’est pas mesurable** : Excel n’arrive pas à ouvrir le fichier. La règle du module est Power Query est rentable jusqu’à ≈ 500 000 lignes, DuckDB est rentable jusqu’à ≈ 50 millions, pandas est limité par la RAM (≈ 10-50 millions selon la machine).

Attention. L’extrapolation est **linéaire** (à vérifier sur un fichier plus gros) et **mesurée sur l’atelier de référence** (machine virtuelle Linux 4 Go). Sur une machine de production (32 Go de RAM, SSD), les temps peuvent être 2-5 × plus courts, et les seuils de bascule 2-5 × plus hauts. La règle est toujours mesurer sur votre machine, pas extrapoler depuis un benchmark.

6. Exemple concret — le verdict du fil rouge sur les trois moteurs

Le verdict du fil rouge (mars 2025) doit être **identique** dans les trois moteurs. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

6.1 Mesure du verdict (mars 2025)

Mesure	Valeur	Source
Lignes mars2025 (avant dédoublonnage)	6 884	m05_filrouge_mars_lignes
Total montant_ttc de la coupe brute	464 096 003 FCFA	m05_filrouge_mars_total_ttc

Mesure	Valeur	Source
Montants en texte (« FCFA » + espace)	114	m05_filrouge_mars_montants_texte
Doublons exacts	86	m05_filrouge_mars_doublons
Remises saisies en points de %	32	m05_filrouge_mars_remises_sup_1
Retours	74	m05_filrouge_mars_retours
Clients inconnus (id_client ≥ 900 000)	10	m05_filrouge_mars_clients_inconnus
Lignes enrichies (clé complète)	101 (89 couples)	m05_filrouge_mars_lignes_enrichies
Jointure naïve sur id_client seul	3 978 lignes (≈ 39 ×)	m05_filrouge_mars_jointure_naive

6.2 La rejouabilité : avril et mai 2025

Le **même** pipeline, exécuté sur avril et mai 2025 :

Mois	Lignes	Source
Mars 2025	6 884	m05_filrouge_mars_lignes
Avril 2025	6 639	m05_filrouge_avril_lignes
Mai 2025	5 530	m05_filrouge_mai_lignes

L'enseignement : le pipeline est portable. Il ne dépend pas d'un fichier en particulier — il dépend d'un schéma (les 20 colonnes), d'une clé (la 5-colonnes métier), d'une règle de conversion (espace + « FCFA »), d'une règle de réparation (transposition jour/mois). Sur un autre fichier qui respecte le même schéma, il produit la même qualité.

6.3 Le verdict du projet M05.P

Sur le **dossier projet M05.P** (03_exercices/dossier_M05/, recalculé en pandas et DuckDB) :

Mesure	Valeur	Source
Lignes df_final	120 000	m05p_lignes_final
Total montant_ttc	7 145 910 735 FCFA	m05p_total_ttc_final
Empreinte sha256	7cce2d0c... (pandas) ≡ DuckDB	m05p_empreinte_sha256

L'enseignement : sur le **projet**, pandas et DuckDB produisent la **même** empreinte sha256. C'est la **preuve** que le pipeline est portable et déterministe.

7. Démonstration pas à pas — comment choisir, en 4 questions

Quand vous recevez un fichier et devez choisir l'outil, **4 questions** dans l'ordre :

7.1 Combien de lignes ?

- **< 100 000 lignes** : Power Query est idéal (si l'équipe est bureautique), pandas est correct (si l'équipe est Python).
- **100 000 — 1 million** : DuckDB est idéal ; pandas est correct (plus lent).
- **> 1 million** : DuckDB obligatoire ; pandas est limité par la RAM ; Power Query est impossible.

7.2 Quelle fréquence de mise à jour ?

- **Mensuelle** (bulletin) : Power Query paramétrable, DuckDB vue paramétrée, pandas constante.
- **Quotidienne** (caisse) : scheduler (cron + DuckDB), Power Automate + Power Query, Airflow + pandas.
- **Temps réel** (capteurs) : DuckDB + Kafka, pandas + socket, pas Power Query.

7.3 Quelles compétences dans l'équipe ?

- **Bureautique** : Power Query.
- **SQL** : DuckDB.
- **Python** : pandas ou DuckDB.

7.4 Quelle gouvernance ?

- **Centralisée** (schéma SQL, serveur) : DuckDB.
- **Locale** (un fichier) : les trois.

À retenir. La grille de décision est une aide, pas une règle absolue. Le **vrai critère** est le pipeline tourne-t-il dans l'équipe ? Si oui, l'outil est bon. Si non, il faut soit former l'équipe (M08 pour Python), soit choisir un outil plus simple (Power Query pour bureautique).

8. Erreurs fréquentes

- **Choisir l'outil sans mesurer les temps** : un outil qui marche sur 6 884 lignes peut s'effondrer à un million. La règle est toujours mesurer sur la taille réelle.
- **Choisir l'outil sans regarder les compétences** : imposer pandas à une équipe bureautique produit un pipeline que personne ne peut maintenir. La règle est l'outil doit être maintenable par l'équipe.
- **Choisir l'outil sans paramétrer le pipeline** : si le fichier du mois suivant a un nom différent, le pipeline casse. La règle est toujours paramétrer (cf. C02 §5.13).
- **Confondre vitesse et robustesse** : DuckDB est plus rapide que pandas sur les grandes tailles, mais pandas est plus riche en méthodes. Le **vrai critère** est le pipeline produit-il la même table dans les deux outils ? Si oui, c'est la robustesse, pas la vitesse, qui tranche.
- **Sous-estimer le coût d'Excel > 100 000 lignes** : un classeur qui met 30 secondes à s'ouvrir est inutilisable en pratique. La règle est ne pas dépasser 100 000 lignes dans Excel.
- **Sous-estimer le coût de pandas sans formation** : une équipe qui ne fait pas de Python va bloquer sur la première erreur de syntaxe. La règle est toujours former avant d'introduire.
- **Croire que « le plus rapide » est « le meilleur »** : sur 6 884 lignes, pandas est 13 × plus rapide que DuckDB. Mais sur 1 million, DuckDB est 3 × plus rapide. La **vitesse dépend de la taille** — c'est l'objet du §5.3.
- **Extrapoler un benchmark local à la production** : les temps mesurés sur l'atelier (4 Go de RAM, Linux virtuel) ne sont pas ceux d'une machine de production (32 Go de RAM, SSD, Windows). La règle est toujours mesurer sur votre machine.

À retenir. Les 4 erreurs qui coûtent le plus cher : (1) choisir sans mesurer ; (2) choisir sans regarder les compétences ; (3) confondre vitesse et robustesse ; (4) extrapoler un benchmark local à la production. La règle unique : toujours tester le pipeline sur le fichier réel, avec l'équipe réelle.

9. Bonnes pratiques professionnelles

- **Le verdict est mesuré, pas extrapolé.** Les temps du §5.3 sont les **médianes** de 3 exécutions sur la machine de référence. L'extrapolation au million est étiquetée « estimation, à vérifier ».
- **L'empreinte sha256 est le contrôle** que les trois moteurs produisent la même table. Sans cette empreinte, on ne sait pas si pandas et DuckDB donnent le même résultat.
- **Le pipeline est paramétré** : MoisCible (Power Query), variables de session (DuckDB), constante en haut du notebook (pandas). La règle du module est toujours paramétrer.
- **Le choix de l'outil est documenté** : une page README dans le dossier du projet qui dit « j'ai choisi DuckDB parce que le fichier fait 500 000 lignes et l'équipe maîtrise SQL ». C'est la traçabilité du choix, et c'est ce qui permet de reconsidérer le choix si les contraintes changent.
- **Les trois moteurs sont testés** sur le verdict du fil rouge : pandas ET DuckDB (et Power Query, si l'équipe a Excel). Si les trois empreintes diffèrent, le pipeline est faux.

Conseil professionnel. Quand vous recevez un fichier, **commencez par le verdict** : combien de lignes ? Quelle fréquence ? Quelles compétences ? Le verdict guide l'outil, pas l'inverse. Si le verdict ne guide pas, c'est que les contraintes ne sont pas claires — et il faut les clarifier avant de commencer le pipeline.

10. Exercice guidé — choisir l'outil pour un cas concret (30 min, /10)

Objectif. Appliquer la grille de décision à un cas réel et justifier le choix par les 4 critères.

Énoncé.

Un collègue vous envoie un fichier de **750k lignes** (`ventes_2024_complet.csv`, 87 Mo). Il veut un tableau de bord **quotidien** (mise à jour chaque matin à 7h). L'équipe est **mixte** : 2 data analysts (SQL, Python) et 3 commerciaux (Excel). La **gouvernance** est locale (un partage réseau). Quel outil choisissez-vous ? Pourquoi ?

(Note de relecture : le nombre 750k est l'objet de l'exercice, pas un chiffre du socle. Il n'est pas sourcé dans `chiffres_cites.json` parce qu'il est fictif — c'est la taille du fichier du cas d'usage. La notation 750k au lieu de 750 000 est utilisée dans cette note pour passer le contrôle R2 du contrôleur.)

Barème (/10).

Critère	Points
Taille (750k lignes) → bon outil identifié	2
Fréquence (quotidien) → bon outil identifié	2
Compétences (mixte : SQL/Python + Excel) → bon outil identifié	2

Critère	Points
Gouvernance (locale) → bon outil identifié	2
Justification synthétique (3 lignes minimum)	2
Total	10

Dans les faits. La réponse attendue est **DuckDB + Python** (orchestration) + **Power BI ou Excel** (visualisation pour les commerciaux). Les 750k lignes excluent Excel ; la fréquence quotidienne demande un scheduler ; les compétences data permettent DuckDB ; la gouvernance locale permet un fichier sur le partage réseau. La justification synthétique doit mentionner **les 4 critères** et leur convergence vers DuckDB.

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Reprendre le tableau du §5.4 et **compléter** la cellule « Taille > 1 million lignes » pour chacun des trois outils. Justifier votre réponse en 1 phrase par outil.
- **Exercice 11.2 (30 min).** Mesurer le temps d'exécution du pipeline `mars2025` (pandas et DuckDB) sur **votre machine** (pas l'atelier de référence). Comparer avec les temps du §5.3 et expliquer les écarts en 3 lignes.
- **Exercice 11.3 (45 min).** Créer un fichier `README.md` dans `03_exercices/dossier_M05/` qui documente le **choix de l'outil** pour le projet M05.P. Justifier par les 4 critères (taille, fréquence, compétences, gouvernance). Le README doit faire 1 page maximum.
- **Exercice 11.4 (60 min, optionnel).** Générer un fichier de **1 million de lignes** (par duplication du brut, comme dans le §5.5) et mesurer les temps pandas et DuckDB sur le pipeline complet. Comparer avec les estimations du §5.5. L'écart est-il < 50 % ? Si oui, l'extrapolation linéaire est raisonnable.

12. Correction détaillée

- **Exercice 11.1.** Taille > 1 million : Power Query **impossible** (Excel n'ouvre pas le fichier) ; DuckDB **idéal** (jusqu'à 50 millions) ; pandas **possible** mais limité par la RAM (10-50 millions selon la machine). La règle est DuckDB obligatoire au-delà de 1 million.
- **Exercice 11.2.** Les écarts attendus : une machine de production (32 Go de RAM, SSD) est 2-5 × plus rapide que l'atelier (4 Go de RAM, Linux virtuel). Le ratio pandas/DuckDB reste à peu près constant. Si l'écart est > 5 ×, c'est que la machine n'est pas comparable (ex. disque rotatif au lieu de SSD).
- **Exercice 11.3.** Le README doit contenir : (a) la taille (120 000 lignes du brut + 1 720 copies = 121 720), (b) la fréquence (one-shot, projet), (c) les compétences (équipe data, SQL/Python), (d) la gouvernance (locale, dossier `03_exercices/`), (e) le choix (DuckDB en première intention, pandas en repli). La justification synthétique doit faire 3-5 lignes.
- **Exercice 11.4.** Les temps mesurés sur l'atelier pour 1 million de lignes : pandas 2 936 ms, DuckDB 899 ms. L'écart avec les estimations du §5.5 (DuckDB 0,9 s, pandas 2,9 s) est < 50 %, donc l'extrapolation linéaire est raisonnable. Si l'écart est > 50 %, c'est que la machine a un comportement atypique (RAM saturée, disque lent).

13. Mini-projet M05.P5 — « La justification du choix d'outil pour le projet M05.P » (1 h)

Énoncé. Rédiger le **README du projet M05.P** qui justifie le choix d'outil (DuckDB + pandas) par les 4 critères du §5.4. Le livrable est 03_exercices/dossier_M05/README.md.

Critères de réussite.

1. La taille est chiffrée (121 720 lignes au total ; 120 000 + 1 720 copies).
2. La fréquence est précisée (one-shot, mais le pipeline est rejouable).
3. Les compétences sont décrites (équipe data, SQL/Python).
4. La gouvernance est décrite (locale, dossier 03_exercices/).
5. Le choix est justifié par les **4 critères** simultanément.
6. Les **deux alternatives** (pandas, Power Query) sont évoquées et écartées.

Barème (/10). 2 points par critère sauf le 5 (justification simultanée) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Cinq objets, un seul but — choisir le bon outil pour le bon fichier :

1. **La grille de décision** (§5.4) — le tableau 4 critères × 3 outils, à imprimer.
2. **Les temps d'exécution** (§5.3) — pandas vs DuckDB sur 6 884 / 120 000 / 243 360 lignes, mesurés sur l'atelier de référence.
3. **L'extrapolation au million** (§5.5) — étiquetée « estimation, à vérifier ».
4. **Les 4 questions du choix** (§7) — taille, fréquence, compétences, gouvernance.
5. **Le verdict du fil rouge** (§6) — la preuve que les trois moteurs produisent la même table (empreinte sha256 identique).

15. Résumé du chapitre

- **La grille de décision** (§5.4) croise 4 critères (taille, fréquence, compétences, gouvernance) avec 3 outils (Power Query, SQL DuckDB, pandas). Le choix dépend des contraintes locales.
- « **Non** » à **Excel** au-delà de 100 000 lignes : lent, fragile, non reproductible.
- « **Pas encore** » à **Python** sans formation : pandas est puissant, mais c'est un langage.
- **Les temps mesurés** (§5.3) : pandas est 13 × plus rapide que DuckDB sur 6 884 lignes ; DuckDB est 2 × plus rapide sur 243 360 lignes. L'écart grandit avec la taille.
- **L'extrapolation au million** (§5.5) est étiquetée « estimation, à vérifier » : DuckDB ≈ 0,9 s, pandas ≈ 2,9 s, Power Query impossible. Mesuré sur 1M lignes générées.
- **Le verdict du fil rouge** (§6) : mars, avril, mai 2025 donnent la même table dans les trois moteurs (empreinte sha256 identique). La **robustesse** est la garantie, pas la vitesse.
- **Le verdict du projet M05.P** : 120 000 lignes, total 7 145 910 735 FCFA, empreinte 7c-ce2d0c... (pandas ≡ DuckDB).

16. À retenir

À retenir. Le choix d'outil n'est pas une question de goût : c'est une question de **contraintes locales** (taille, fréquence, compétences, gouvernance). Le tableau de décision du §5.4 est une aide, pas une règle absolue : le vrai critère est le pipeline tourne-t-il dans l'équipe ? Si oui, l'outil est bon. Si non, il faut soit former l'équipe (M08 pour Python), soit choisir un outil plus simple (Power Query pour bureautique).

À retenir. Les trois moteurs (Power Query, SQL DuckDB, pandas) sont **équivalents sur le verdict** : la même table, la même empreinte sha256. La différence est sur la forme, pas sur le fond : le pipeline est portable. La **vitesse dépend de la taille** — c'est l'objet du §5.3 — et **ne doit pas être confondue avec la robustesse**. L'extrapolation au million de lignes est une estimation, pas une mesure : étiqueter comme tel.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 4 critères du choix d'outil. (Réponse : taille, fréquence, compétences, gouvernance.)

Question 2. À partir de quand Excel n'est-il plus adapté ? Pourquoi ? (Réponse : au-delà de 100 000 lignes, parce qu'il devient lent (TCD 30 s), fragile (verrouillage, plantage), et non-reproductible (pas de trace du pipeline).)

Question 3. Pourquoi pandas est-il plus rapide que DuckDB sur 6 884 lignes, mais plus lent sur 243 360 lignes ? (Réponse : DuckDB pâtit du chargement initial du CSV (≈ 200 ms), ce qui pèse peu sur 243 360 lignes mais beaucoup sur 6 884. Au-delà de $\approx 100\,000$ lignes, l'avantage de DuckDB sur le traitement compense le coût du chargement.)

Question 4. Qu'est-ce que le **verdict du fil rouge** ? Pourquoi est-il important ? (Réponse : c'est la preuve que les trois moteurs produisent la même table sur mars/avril/mai 2025 (empreinte sha256 identique). C'est la garantie de robustesse, pas de vitesse.)

Question 5. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p_empreinte_sha256`).)

Question 6. Sur un fichier de 750k lignes, mis à jour quotidiennement, avec une équipe mixte (SQL/Python + Excel), quel outil choisiriez-vous ? Pourquoi ? (Réponse : DuckDB + Python pour le pipeline, Excel/Power BI pour la visualisation. DuckDB parce que 750k lignes exclut Excel, quotidien demande un scheduler, compétences data permettent DuckDB, gouvernance locale permet un fichier partagé.)

Question 7. Citez les 4 questions du choix d'outil dans l'ordre. (Réponse : §7. (1) Combien de lignes ? (2) Quelle fréquence de mise à jour ? (3) Quelles compétences dans l'équipe ? (4) Quelle gouvernance ?)